



tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA
Escuela Superior de Ingenieros · Ingeniarien Goi Mailako Eskola

● C++ Laboratory Lessons

Lesson N° 8

C++
PARA INGENIEROS SUPERIORES

Computer Programming II

Dr. Paul Bustamante

OUTLINE

OUTLINE.....	1
1. Introduction.....	1
1.1 Exercise 1: Class Fraction v1.0	1
1.1.1 Simplifying the fractions.....	2
1.2 Exercise 2: Class Student	2
1.3 Exercise 3: Dynamic objects	3
1.4 Exercise 4: Static and dynamic arrays.....	4
1.5 Exercise 5: Friend Functions.....	4
1.6 Exercise 6: Bank Account.....	5

1. Introduction.

In this lab session you will learn how to implement dynamic objects and dynamic arrays with classes. Also, you will practice more with friend functions and static members, inside the classes.

1.1 Exercise 1: Class Fraction v1.0

In this exercise you will create the class **Fraction**, which allows working with fractions as if they were a C++ type of variables, analogous to **int**, **long**, **double**, etc.

The class **Fraction** must have two member variables with **private** access: **int num**, **den**; which will be used to store the **numerator** and **denominator** of the fraction. Below the class **Fraction**, is shown, which must be implemented in **Fraction.h**:

```
class Fraction
{
    int num,den;

public:
    Fraction(int n=1, int d=1);           //constructor
    ~Fraction();                          //destructor

    Fraction add(Fraction& f);
    void prt(){
        cout << "["<<num<<"/"<<den<<"] ";
    }
};
```

Code of **add**, **constructor** and **destructor** functions, to be implemented in **fraction.cpp**:

```
Fraction::Fraction(int n, int d)
{
    num=n; den=d;
    cout << "Constructor Fraction.."<<endl;
}
Fraction::~~Fraction()
{
    cout << "Destructor Fraction.."<<endl;
}
Fraction Fraction::add(Fraction& f)
{
    long dd = den * f.den;
    long nn = num * (dd/den) + f.num*(dd/f.den);

    Fraction fs(nn,dd) ;
    return fs;
}
```

Write the following code in **main**, in order to test the class:

```
//file main.cpp
#include <iostream>
using namespace std;
#include "fraction.h"

void main()
{
    int n,d;
    Fraction f1(1,4);
    cout << "Enter N,D:?"; cin >> n >> d;
    Fraction f2(n,d);

    cout << "\nF1: "; f1.prt();
    cout << "\nF2: "; f2.prt();

    Fraction fs = f1.add(f2);
    cout << "\nF1 + F2 = " ;
    fs.prt();
    cout << "\nFin.."<<endl;
}
```

1.1.1 Simplifying the fractions

Now you must create the **private** function **simplify()** in the class **Fraction** to implement the fractions simplifying algorithm using the **GDC (Greatest common divisor)** See the code:

```
void Fraction::simplify()
{
    long m, n, r;
    if (num<den) { m=den; n=num; }
    else { m=num; n=den; }
    r=m%n;
    while (r != 0)
    {
        m=n;
        n=r;
        r=m%n;
        //r=remainder of m/n
    } //n is the g.d.c.
    num /= n;
    den /= n;
}
```

To test the use of the function, make the call in the constructor and in the **add** function. See next how it is used in the constructor. You should do the same in the **add** function:

```
Fraction(int n=1, int d=1) //constructor
{
    num=n; den=d;
    simplify(); //call to the simplify function
}
```

Use the same **main** program to see the difference. Fractions should appear simplified, i.e. if you introduced 2/4, ½ appear in the console. And if the result of a sum is 4/10, you should get 2/5. If you don't get this, something is going wrong. Ask!

1.2 Exercise 2: Class Student

This exercise consists of making a program to manage “**students**” in a database. For that, you must start with the class **Student** given below. It has to contain just two members functions: **prt()** and **get()**, which will allow to ask for the name and the average mark. You have to implement the functions in the file **student.cpp**.

<pre>// Student.h class Student { char name[20]; double average; public: Student(char *n=(char*)"joe",double a=0) { strcpy(name,n); average = a; } void prt(); void get(); public: ~Student(void){ // put your code here } };</pre>	<pre>// Student.cpp #include "Student.h" void Student::prt() { cout << name << "," << average << endl; } void Student::get() { cout << "Name,Average:?"; cin >> name >> average; }</pre>
--	--

In order to check whether you have written the class correctly, use the following code:

```
//main.cpp
#include <iostream>
using namespace std;
#include "student.h"
void main()
{
    Student s1;
    Student s2((char*)"Ana",6.5);
    Student s3;
    s3.get();

    cout << "Student's List:" << endl;
    s1.prt();
    s2.prt();
    s3.prt();

    cout << "End" << endl;
}
```

1.3 Exercise 3: Dynamic objects

This is an interactive program where the user **must introduce** one by one the objects, which means that they'll be created dynamically at runtime. Change the **main** program with the following code. Compile the project and run it, observing the result.

```
void main()
{
    int n=0;
    cout << "Dynamic objects"<<endl;

    Student *pl[100]; //array of dynamic objects
    pl[n] = new Student((char*)"Ana",6.0); //example how create a new Student
    pl[n]->prt(); //ex. How print the data
    n++;
    int opc;
    while(1){
        cout << "*** Menu **\n1.Add 2.Prt 3.Exit? ";
        cin >> opc;
        if (opc==1){
            //Put your code here to Add
        }else if(opc==2){
            //Put your code here to Print
        }
    }
```

```

        }else{
            break; //exits from while
        }
    }
    cout << "End" << endl;
}

```

1.4 Exercise 4: Static and dynamic arrays

Now you should try to create a fixed number of Students (static array) and a dynamic number of Students, for which you must modify the main program:

```

#include <iostream>
using namespace std;
#include "student.h"

#define NUMS 5
void main()
{
    int num;
    cout << "Static"<<endl;
    Student vec[NUMS];
    for(int i=0;i<NUMS;i++){
        vec[i].get();
    }
    cout << "Data:"<<endl;
    for(int i=0;i<NUMS;i++){
        vec[i].prt();
    }

    cout << "Dynamic"<<endl;
    cout << "Enter number of students:";
    cin >> num;
    Student *pv = new Student[num];
    for(int i=0;i<num;i++) pv[i].get();

    cout << "Data:"<<endl;
    for(int i=0;i<num;i++) pv[i].prt();

    delete [] pv;
}

```

1.5 Exercise 5: Friend Functions

In this exercise you must create a friend function, **passed**, which should return true (TRUE) or false (FALSE) depending on whether the student has passed or not.

```

class Student
{
    char name[20];
    double average;
public:
    Student(char *n=(char*)"none", double a=5.0);
    void prt();
    void get();
    friend bool passed(Student*);
};

```

In the same file where **main** function is defined, add the function in the following way:

```

bool passed(Student *st)
{
    if (st->average >= 5.0 ) return true;
    else return false;
}

```

Now add one more option in the menu within **main** to display on screen the students which have passed.

1.6 Exercise 6: Bank Account

This exercise consists of making use of **static** variables, or class variables, which are the same for all objects. Below the structure of the class **BankAcc** is shown:

```
class BankAcc
{
    double balance;
    static double interest;
public:
    BankAcc (double b=0){ balance=b; }
    void AddBalance(double quant);
    void Payment(double amount);
    void IntPaym();
    double Balance(){ return balance;}
    bool Transfer (double amount, BankAcc & c);
};
```

You must add the necessary code so you can perform the following operations with the accounts: Add Balance, Interest Payment, Payments, and Transfers from one account to another, etc.

```
double BankAcc::interest = 0.01;           //1% per month
void main()
{
    BankAcc acc1(100), acc2(200);
    cout << "Acc1:" <<acc1.Balance() ;
    cout << "Acc2:" <<acc2.Balance()<< endl;

    acc1.IntPaym(); acc2.IntPaym();
    acc1.Payment(50); acc2.Payment(20);
    cout << "Acc1:"<<acc1.Balance()<< "Acc2:"<< acc2.Balance()<<endl;

    if ( acc1.Transfer (80, acc2) == true){
        cout << "Trans. made"<<endl;
    }else{
        cout << "No balance.."<<endl;
        acc1.AddBalance(100);
    }
    cout << "Acc1:" <<acc1.Balance()<<"Acc2:"<< acc2.Balance() << endl;
}
```